

Memoria descriptiva del programa informático

MULTIHERO – LAS TABLAS JUGANDO

Documento preparado a efectos de depósito o documentación de obra (software) ante registros o procedimientos de propiedad intelectual y conexos.

Fecha de la memoria: 26 de abril de 2026

Nota previa: Este texto describe la arquitectura y funcionalidad desde el criterio técnico. El cumplimiento de requisitos formales del registro o depósito (forma, anexos, metadatos, comprobación de autenticidad, honorarios) corresponde al titular y, en su caso, a asesoramiento jurídico. La licencia de terceros (bibliotecas) no implica ceder la obra propia del desarrollador.

1. Identificación de la obra

Campo	Descripción
Denominación	MULTIHERO
Tipo	Aplicación interactiva (cliente) — juego educativo de matemáticas
Plataformas	Web (navegador) y, mediante empaquetado nativo, Android (ID de paquete: `com.caimanint.multihero`)
Lenguaje principal	TypeScript ; interfaz con TSX (React)
Motor de interacción	React 19 con compilación Vite 6
Capa nativa móvil	Capacitor 8 (Android)

****Titular / desarrollo**** (según términos legales del proyecto; completar o sustituir en documento oficial):

Pacual / Pascual Embid García — contacto: pascualembid@gmail.com

2. Finalidad y campo de la obra

MULTIHERO es un ****programa de entretenimiento educativo**** orientado al aprendizaje y refuerzo de ****matemáticas**** (tablas de multiplicar, resolución de operaciones, progresión por mapa, recompensas virtuales) con una interfaz lúdica, personajes, economía de monedas, tienda, inventario y mecánicas de juego (batallas, minijuegos, misiones diarias, modos de retos, multijugador entre usuarios autenticados). La experiencia es ****multilenguaje**** (es, ca, en, fr, y extensiones vía i18n).

El software combina: (a) lógica de juego y estado en el cliente, (b) posible almacenamiento local y (c) servicios en la nube (autenticación, base de datos en tiempo real) cuando el usuario activa cuenta.

3. Arquitectura técnica (visión global)

1. ****Capa de presentación (UI/UX):**** componentes React reutilizables, estilos con ****Tailwind CSS 4**** (Vite + plugin), iconografía (Lucide), animaciones. Flujo por “pantallas” (``Screen``) y estado global de usuario (``UserState``).

2. ****Capa de lógica de negocio / juego:**** módulos en ``src/lib/`` — motor de problemas adaptativos (``engine``), recompensas, tienda, misiones diarias, sonido, celebraciones, utilidades, integración recompensas con anuncios, etc.

3. **Gráficos 3D (parte del juego):** **Three.js** vía **@react-three/fiber** y **@react-three/drei** para escenas del minijuego “Math Hero” y afines.
4. **Internacionalización:** **i18next** y **react-i18next**, con recursos y textos legales en `src/legal/`` (términos y política de privacidad en varios idiomas).
5. **Servicios externos (integración):**
 - **Firebase (Google):** autenticación (email/contraseña, Google, anónima, vinculación con Apple en iOS si procede) y **Realtime Database** para datos vinculados a usuario (p. ej. códigos de amigo, presencia en multijugador).
 - **Capacitor:** puente a APIs nativas; plugins `@capacitor/app``, ``browser``, autenticación Firebase nativa, **AdMob** (`@capacitor-community/admob``) para anuncios y recompensas.
6. **Build y despliegue web:** `vite build`` genera `dist/``. Sincronización a Android: `npx cap sync android`` (assets web en `android/app/.../assets/public``).
7. **Firma y distribución Android:** proyecto Gradle en `android/``; puede firmarse con keystore (configuración en `keystore.properties`` y/o `local.properties`` con parámetros `play.*``).

4. Módulos funcionales principales (descripción no exhaustiva)

- **Mapa y progreso:** desplazamiento por nodos, batallas, estrellas, nodos bonus, rachas, energía, experiencia, monedas.
- **Shop e inventario:** adquisición de artículos virtuales, consumibles, modificadores.
- **Perfil, avatares, ajustes (audio, idioma).**
- **Math Hero / MathHeroScreen y Runner:** secciones de juego 3D o runner según nodo.

- **Minijuego Platform Climb** (`PlatformClimbScreen`` + lógica en `lib/platformClimbGame.ts`` y similares).
- **Multijugador y PvP** (`useMultiplayer`` , pantallas PvP batalla, remolcador, sprint, lista de amigos, códigos de amigo) con sincronización vía Realtime Database y UIDs de Firebase.
- **Misiones diarias** (`DailyMissionHubScreen`` , `dailyMissionWeek`` , bonificaciones con anuncios recompensados).
- **Diploma / logro** final de multiplicación (modal y condiciones de nivel 100).
- **Recompensas con vídeo recompensado** (`rewardedAds.ts`` + SDK AdMob).
- **Aceptación de términos y visor de textos legales** (`LegalDocumentsModal`` , `OnboardingScreen``).
- **Audio:** gestión de sonido y BGM (`lib/audio`` , `mapAudio`` , etc.).

5. Estructura relevante del código fuente (resumen)

Ruta base del repositorio: raíz `mates/`` .

- `src/App.tsx`` — **aplicación principal** (miles de líneas: encaminamiento de pantallas, auth, progreso, mapa, tienda, integración Firebase).
- `src/main.tsx`` — punto de entrada React.
- `src/components/`` — pantallas y componentes (p. ej. `MathHeroScreen`` , `PlatformClimbScreen`` , `Pvp*`` , `OnboardingScreen`` , `WelcomeSplash`` , modales).
- `src/hooks/`` — p. ej. `useMultiplayer.ts`` .
- `src/lib/`` — motor, audio, recompensas, Firebase shim Capacitor, utilidades, i18n base.
- `src/types.ts`` — tipos TypeScript (pantallas, estado de usuario, tienda, etc.).
- `src/constants.ts`` — constantes de juego.
- `src/legal/`` — textos legales multilingües; `legalContent.ts`` resuelve idioma.

- `public/`` y `index.html`` — activos estáticos y plantilla.
- `android/`` — proyecto Android (Gradle, manifiestos, `MainActivity``, assets de Capacitor).
- `legal-web/`` — HTML estáticos (privacidad, términos, eliminación de cuenta) para enlace en tiendas.
- `package.json`` — dependencias y scripts (build, `android:sync``, `android:release``).

Fichero de **licencia** del código (donde se indique): comprobar en el repositorio (p. ej. cabeceras SPDX en archivos, si existen). Las dependencias de terceros se rigen por sus respectivas licencias (MIT, Apache-2.0, etc. en `node_modules`` y documentación de npm).

6. Datos, privacidad y terceros (resumen)

- El programa puede almacenar **estado de juego y preferencias** en almacenamiento local del dispositivo/navegador.
- Con **sesión de usuario** (Firebase), pueden tratarse **identificadores de cuenta, email según proveedor, progreso y datos vinculados** descritos en la **Política de privacidad** (texto en app y, si aplica, en `legal-web/privacidad.html``).
- **Publicidad y Google AdMob** pueden implicar identificadores y datos según políticas de Google.
- No constituye el objeto de esta memoria el detalle jurídico; debe coincidir con la declaración en **Google Play (Seguridad de los datos)** y con los textos legales publicados.

7. Versiones, compilación e integridad (referencia)

- **Número de versión** del paquete npm interno: ver ``version`` en ``package.json`` (p. ej. 0.0.0; el desarrollador puede fijar versión semántica para el depósito).
- **Versionado Android** (``versionCode`` / ``versionName``): ver ``android/app/build.gradle`` (valores concretos en el momento de generar el binario a depositar).
- Para acompañar un depósito, suele aportarse el **código fuente** o un **fichero comprimido** y/o un **resumen criptográfico (hash SHA-256)** del artefacto entregado, según requisitos del registro; generar al cerrar la versión a registrar.

8. Declaración de originalidad (propósito documental)

La obra descrita (código fuente, diseño de flujos, textos y medios no licenciados de terceros, selección y coordinación) constituye un programa de **autoría del titular** o de quien se acredite en el contrato de desarrollo, salvo partes bajo licencia de terceros usadas de conformidad con sus términos. Las marcas, logotipos y marcas de terceros (Google, etc.) quedan sujetas a sus titulares.

9. Ejecutable y límite de tamaño (30 MB) — justificación

El procedimiento solicita el **programa en formato ejecutable** dentro de un fichero **ZIP**, con **tamaño máximo 30 MB**, o en su defecto **justificación en la memoria** si no se aporta.

Naturaleza del ejecutable de MULTIHERO

- **Versión Android:** el instalable es un **APK** (o AAB en Play) generado tras ``npm run build``, ``npx cap sync android`` y compilación Gradle (``assembleRelease``). El binario firmado **reúne** la aplicación web empaquetada, el runtime Capacitor, los plugins (Firebase, AdMob, etc.) y los recursos gráficos y de audio; su **tamaño típico** es del orden de 100 MB o superior, muy por encima del límite de 30 MB.

- **Versión web empaquetada:** la carpeta de salida ``dist/`` tras ``vite build`` (recursos estáticos + JavaScript minificado + medios) también **supera de forma habitual los 30 MB** en este proyecto, por el volumen de **gráficos**, modelos/escenas 3D, sonidos y recursos del juego educativo.

Por qué no se adjunta un ZIP de ejecutable ≤ 30 MB

Incluir un ejecutable **completo y funcional** en un único ZIP bajo **30 MB** no resulta técnicamente viable **sin mutilar** la obra (eliminar niveles, audio, texturas, funcionalidades o bibliotecas esenciales), lo que **no** representaría fielmente el programa objeto de protección.

Qué se aporta en su lugar (coherencia con el depósito)

- **Código fuente completo** en el ZIP correspondiente, con instrucciones de compilación (``npm ci`` o ``npm install``, ``npm run build``, sincronización Capacitor y generación del APK con Android Studio / Gradle), de modo que el **ejecutable puede reproducirse** a partir del material depositado con herramientas estándar y gratuitas.

- Identificación del **identificador de aplicación** en Google Play: ``com.caimanint.multihero``, como prueba de la forma materializada en plataforma de distribución (si el registro admite referencia externa o captura).